玩转OpenClaw，你需要了解的：核心架构、运作原理、Agent部署步骤

原创 腾讯程序员 腾讯技术工程 2026年3月9日 02:26 广东 631人



作者：冰以东

> 建议有打算深入了解OpenClaw的同学优先看OpenClaw源码和官方文档，目前该项目正在高频迭代中。本文重点从核心框架、通信机制进行介绍，争取让你看完本文后知道OpenClaw是怎么运作的，以及其能力边界在哪里。以及尤其希望大家注意的，是OpenClaw的安全风险，如果选择部署OpenClaw，就按最坏的打算(数据全Open)去对待自己机器上的数据。

前言

（一）OpenClaw 到底有什么不同?

最近半年AI领域的产品层出不穷，在OpenClaw爆火之前比较类似的产品是Claude的Happy(一款Claude code ssh软件)。

喜欢使用happy的朋友经常问我的一句话是:"OpenClaw那么好玩吗？跟Happy有什么区别？"

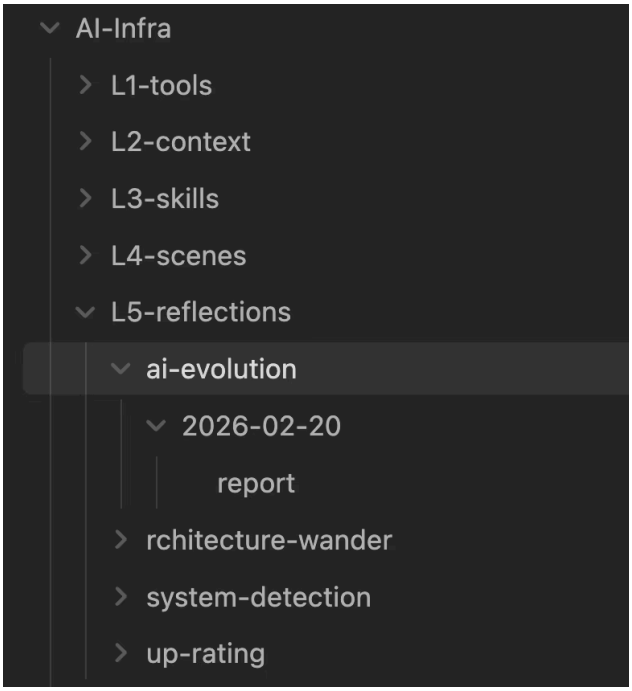
当初看到OpenClaw的功能之后我立马被吸引住，在面对这个问题后，我正好从抽象层面总结一下OpenClaw的优势所在。

1、OpenClaw 技术框架不复杂，它的优势在于共识的推广

作为程序员，为了让大家直观理解OpenClaw的项目架构强度。 在看完OpenClaw框架后，我先斗胆做个类比，大概说一下OpenClaw的技术难度： 大概就类似AI Coding诞生前，具备「初级推荐算法的前后端通信App」的难度。

做过几年开发的同学都知道，这其实并不难，所以技术框架并不是OpenClaw的亮点。

OpenClaw的优势在于共识的推广，举个具体的例子，在没有OpenClaw之间，我们基本人手一个自己搭建的Agent，像我之前搭建的 L1~L5 5层架构Agent：



相信每个搭Agent架构的同学，都得考虑skills管理、Agent身份赋予、Agent架构自进化、memory-search和Session管理这些。

这就导致一个问题：每次我跟朋友交流Agent之前，都是要先简单介绍一下各自Agent的架构，然后再聊具体的落地Case，Session、memory管理的方案，可能都得先聊半天。

但 OpenClaw 把Agent架构推广之后，我们基于OpenClaw搭建个人Agent后，就不用再介绍Agent架构是什么了，我们再聊的话题就是：怎么保活、怎么进一步替换rag算法库、怎么部署多Agent、怎么应用good case。

简直丝滑爆了。

2、多Agent的天然支持

熟悉LLM底层原理的同学都知道，LLM成也transformer，目前卡脖子的地方，也在transformer。Context的瓶颈严格约束了单Agent智能的发挥。

传统Prompt定义Agent身份，再加上层出不穷的skills，正在一步一步蚕食LLM的Context窗口。

为了更好使用LLM，专事专做似乎已经变成更好使用Agent的共识。

3、做和AI能力正交的事情

AI时代，选择不做什么事情和选择做什么事情一样重要。

一年前跟Manus的朋友聊天时，当时他就分享过一个观点：要做和AI能力正交的事情。

花时间精力打造和迭代自己的Agent，其实就是跟AI能力正交的一件事，跟培养一个人一样，他可以是很聪明，但他认知世界和做事的能力，需要我们来教导他，这是千人千面的一个话题。

当AI模型越来越聪明，我们只需要升级Agent使用的底层LLM即可，那些跟AI交互留下来的长期数据，都将会变成我们未来更好驱动AI的私人宝贵数据。

（二）我想部署OpenClaw，必须要买机器吗？

买机器不是必须的，目前使用OpenClaw有2种主流方案：云机 和 自部署。

下面我按顺序介绍一下。

1、云机

腾讯云官网现在已经支持云机部署OpenClaw，也十分便捷。

如果能接受云机的操作习惯，这种方式是最合适的，不需要用实体机，数据也可以随时download下来。

2、自部署

接下来介绍一下自部署，这可能是大家最关心的。

（1）Mac 和 Windows 怎么选?

OpenClaw的部署和诸多工具，对Mac环境天然友好。如果可以，最好选Mac。Windows当然也可以部署，就是折腾一些，网上都有教程，这里就不展开说了。

（2）Mac的配置怎么选?

因为OpenClaw部署之后基本就在本地运行着，所以Mac系列优先推荐Mac Mini。

其次关于 Mac Mini的配置，这里主要涉及3个指标：芯片、内存和磁盘。

a. 芯片

从软件适配出发，既然都采用Mac Mini的方案了，那么至少要是M系列的芯片。

其次，M1~M4，要看部署时的诉求，如果平时不需要跑文生图、文生视频的本地模型，那么M1芯片是性价比最高的。OpenClaw运行时基本都只是调各种api，不怎么吃性能。

b. 内存

内存的诉求也是看是否需要本地部署文生图、文生视频的模型，如果需要部署模型，还是建议24g内存。

c. 磁盘

磁盘的诉求也是看是否需要本地部署文生图、文生视频的模型，ComfyUI上的模型，动辄都是30G起步，所以如果用来跑本地模型，至少256G起步。

目前 Mac Mini M1 1TB 市场价3K左右，MacMini M4 512g大概7k左右。

不成熟的建议就是：既然都打算部署本地模型了，建议多花4K玩个大满配，以后给家里小朋友做文生图、文生视频，不用付费买API，还是很香的。

（三）OpenClaw 推荐使用哪个IM工具？

这是容易踩坑的地方，筛选IM工具，我总结下来有3个原则：安全性、可用性、易用性。

我这里不做推荐，只是建议大家从这3个原则里，明确下自己的诉求，来做决策。

1、安全性

从数据隔离的大前提出发，一定要避免**个人误操作**发送了一些私密内容给OpenClaw（注意：**千万不要**把OpenClaw当成「文件传输助手」），以及严防**习惯OpenClaw**之后忽略了其安全风险，时刻提醒自己部署OpenClaw之后，就是在跟把机器人公开在网上没什么区别，按最坏的情况去预估风险。

2、可用性

这是最容易踩坑的一点，如果你只是部署单Agent，你会发现还是够用的。

但当你部署多Agent之后，你的IM调用额度会飞速消耗，我在部署10个Agent之后，IM的额度直接耗尽，原因是OpenClaw的网关机制中有一个定时快照的逻辑，

```
const healthInterval = setInterval(() => {
 void params.refreshGatewayHealthSanpshot({probe:true})…
})
```

该快照逻辑会ping IM，60秒一次，如果Agent很多，IM额度将很快被消耗完。

所以如果你有多Agent的诉求，你需要选一个额度多(无额度约束)的IM。

3、易用性

相比OpenClaw推荐的国外IM软件，国内的软件易用性还是高一些。

（四）OpenClaw的配置麻烦吗？

如果只是把工程run起来 + 配置IM机器人，半小时足矣。

但如果是配多Agent，配不同Agent的任务分配、调试Skills、定制Agent身份、调试定时任务、自部署模型，那还是比较耗时的，根据 个人经验+网上交流经验+身边小伙伴配置 的经验，大概需要2-3天的时间。

（五）折腾OpenClaw，我能有什么收益？

1、技术层面的收益

完整搭建完这套流程后，对Skills的理解、对多Agent的理解、对自部署模型的理解、对memory-search原理的理解、对Agent经典架构的理解，都可以上一个层次。

比如这些问题：

”如果让你设计一个Agent，它的长短期记忆链路你打算怎么设计？”

“如果让你设计一个多Agent架构，你会设计哪些通信方式？”

“中大型项目中，怎么对多Skills的情况进行管理，怎么避免多Skills、低质Skills爆炸的问题？”

“memory-search方案的原理是什么，一个完整的LLM对话是怎么在transformer框架中流转的?”

都可以在折腾OpenClaw中自己摸索到，更进一步的，面对当前各种Vibe的Agent项目，可以比较清晰的看出这类项目的含金量和生命周期，这对我们在AI时代做技术方向的判断，都是至关重要的。

2、个人Jarvis

这个是搭建工具之后，通过发挥个人创意可为自己带来的增益。

简单来说，就是你拥有了个人Jarvis，你可以用他定制任何你想让它进行的任务，本文会在「**三、Good Case 分享**」分享几个我自己实际用下来不错的case。



腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

688篇原创内容

公众号

一、OpenClaw 多Agent部署

（一）OpenClaw的quick-start

1、快速完成OpenClaw的部署工作

（1）部署前需要提前准备的

[1] 选择你的IM平台

不同IM平台有不同的配置方式，这里按个人喜好选择就好。

[2] 选择你的LLM Model

LLM Model越聪明，OpenClaw发挥就会越好、越稳定。

（2）本机部署OpenClaw

```
curl -fsSL https://openclaw.ai/install.sh | bash
```

使用如上命令安装OpenClaw后，会自动启动 **openclaw onboarding**，即进入OpenClaw main Agent的配置阶段，配置阶段按导引进行配对即可，其中config skills的部分可以跳过，后续自己专门配置即可。

注意：

十分不建议使用Claude Code来进行OpenClaw的配置，CC目前对OpenClaw项目配置的了解还有偏差，极大可能出现”it works,but it also broke”的情况，导致工程后续极难维护，建议还是阅读OpenClaw官方文档，使用OpenClaw的官方指令配置Agent。

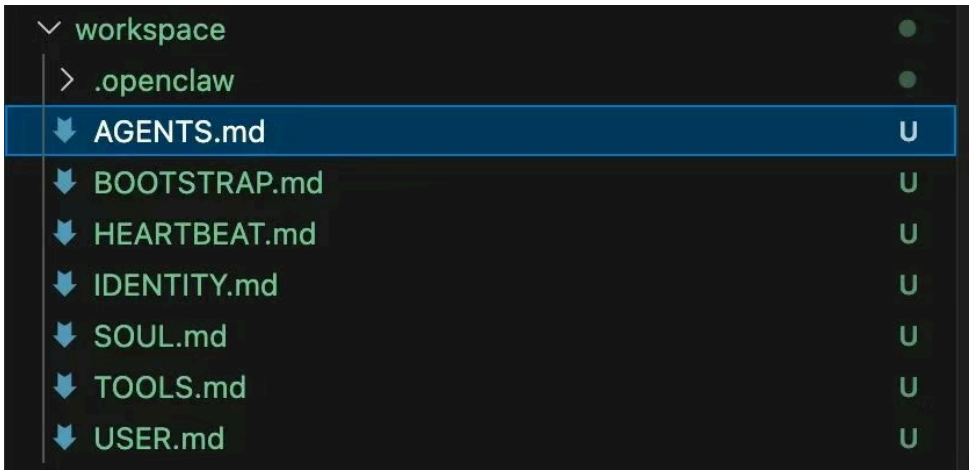
不过，使用CC来辅助Debug OpenClaw还是很高效的，推荐流程：

Download OpenClaw源码到本地 → 让CC先学习下OpenClaw的源码 → 再让CC针对本地OpenClaw的配置进行debug。

给够CC充足源码语料后，交付的水平会大幅提升。

（二）OpenClaw Agent核心架构介绍

1、Agent的核心配置项



每个Agent都有其对应的workspace，如图是一个Agent最核心的配置文件。其各文件的含义如下：

```
AGENTS.md: Agent职责声明，决定工具权限

SOUL.md: 个性化提示词，注入system prompt

TOOLS.md: 工具白名单/黑名单，安全边界

IDENTITY.md: 身份标识（name/avatar），通道展示

USER.md: 用户偏好，上下文先验

HEARTBEAT.md: 定时任务配置（可选）
```

```
BOOTSTRAP.md: 首次onboarding引导（一次性消费）

MEMORY.md: 用户记忆文档（RAG源）
```

通过查看OpenClaw的源码，我们可以找到这么一个方法：

```
src/agents/workspace.ts:498-555

exportasyncfunction loadWorkspaceBootstrapFiles(dir: string): Promise<WorkspaceBootstrapF
const entries = [
    { name: "AGENTS.md", filePath: path.join(resolvedDir, "AGENTS.md") },
    { name: "SOUL.md", ... },
    { name: "TOOLS.md", ... },
    { name: "IDENTITY.md", ... },
    { name: "USER.md", ... },
    { name: "HEARTBEAT.md", ... },
    { name: "BOOTSTRAP.md", ... },
  ];
// + 动态检测 MEMORY.md / memory.md

for (const entry of entries) {
    const loaded = await readWorkspaceFileWithGuards({...});
    // openBoundaryFile: 检查inode/dev/size/mtime，防止路径穿越
  }
}
```

文件顺序即优先级。AGENTS定义能力边界，SOUL注入灵魂，TOOLS划定禁区，这8个文件构成Agent的完整人格。

在这里，我大力推荐朋友们阅读 **AGENTS.md** 这个文件⭐⭐⭐，这个文件详细介绍了一个Agent的启动、memory管理的流程，自我感觉堪称OpenClaw最核心的Prompt文件。

2、Agent 的启动流程

Agent不是常驻进程，而是per-session的瞬态实例。每个对话都是一次完整的加载-执行-销毁循环。

通过查看OpenClaw的源码，我们可以找到这么一个方法：

```
src/agents/pi-embedded-runner/run/attempt.ts:1-116

import { createAgentSession, SessionManager } from"@mariozechner/pi-coding-agent";

// 1. 加载bootstrap上下文
const bootstrap = await resolveBootstrapContextForRun({...});
```

```
// 2. 创建SessionManager（封装Pi Agent的session持久化）
const sessionManager = await prepareSessionManagerForRun({
  sessionKey,
  workspaceDir,
  ...
});

// 3. 构建system prompt（注入workspace文件内容）
const systemPrompt = buildEmbeddedSystemPrompt({
bootstrapFiles: bootstrap.files,
  ...
});

// 4. 创建agent session
const agentSession = await createAgentSession({
  model,
  systemPrompt,
  sessionManager,
tools: createOpenClawCodingTools({...}),
  ...
});
```

可以看到，System Prompt是动态生成的，即每次run都会重新读取workspace文件，确保配置实时生效。

（三）OpenClaw Agent的记忆力机制

了解OpenClaw Agent的记忆力机制，对于我们后续精细化对Agent优化、管理skills、学习Agent记忆力机制设计方式，都有很大的帮助。

和OpenClaw记忆力机制相关的配置有三个：

```
// 会话信息
.openclaw/agents/ceo/sessions/xxxx.jsonl

// 按天记录的memory
.openclaw/workspace-ceo/memory/YYYY-MM-DD.md

// LLM精练后的memory
.openclaw/workspace-ceo/MEMORY.md
```

通过查看 **.openclaw/agents/ceo/sessions/xxxx.jsonl** 文件，我们可以发现该.jsonl文件记录了会话的记录。

但这时我们会产生如下问题：

1、Session是怎么实现会话按需加载的？

正如我们前面所说的，不同的会话有其对应的.jsonl。

（1）Session Header

在.jsonl 文件中，Session Header 是 JSONL 文件的第一行元数据，结构如下：

```
{
    type: "session",
    id: "session-uuid",           // sessionId
    cwd: "/path/to/workspace",    // 工作目录
    timestamp: 1234567890,
    parentSession?: "parent-id"   // 可选，fork 时存在
  }
```

我实际捞一个Session，可见其Header内容如下：

```
{"type":"session","version":3,"id":"fd292408-168e-4c7b-9f2a-ff7ec0a7c492","timestamp":"20
```

（2）Agent 的 Session 加载

Session 的加载也是懒加载机制，当消息到达路由到 SessionKey 之后，OpenClaw会查找 sessions.json 获取当前 SessionId，将 SessionId 对应的.jsonl 加载到Agent中。

2、Session太长，是不是就挤爆LLM Context了？是怎么优化的？

（1）LLM 感知 Session 的完整流程

OpenClaw显然是不会将全量的Session加载进LLM中，那么它做了什么优化呢？

在Session加载 → LLM 感知阶段，按如下流程进行load：

```
1. 加载 JSONL 文件
2. SessionManager 解析为 messages 数组
3. 应用过滤/裁剪策略
4. 发送给 LLM
```

这里最有意思的就是【3. 应用过滤/裁剪策略】

（2）Session 应用过滤/裁剪策略

A. Compaction（压缩） - 持久化

当对话太长时，旧消息会被 总结 成一个 summary。

比如：

```
压缩前发送给LLM:
[user1, assistant1, user2, assistan2, … ,user100,assistant100]

压缩后发给LLM:
[compaction_summary, user80, assistant80, … ,user100, assistant100]
           ↑
         firstKeptEntryId 来控制compaction程度
```

进行该 compaction 压缩后，会写入 JSONL 文件（也即进行了持久化）。

下次对话继续使用的就是压缩后的历史。

B. Session Pruning（修剪） - 临时

在发送给 LLM 之前，临时裁剪旧的 tool 结果。

比如：

```
修剪前 JSONL 文件中:
[user1, assistant1, toolResult1(10KB), user2, assistant2, toolResult2(5KB)]

修剪后发送给LLM的内容:
[user1, assistan1, "[Old tool result cleared]", user2, assistant2, toolResult2(5KB)]
            ↑ 旧的 tool 结果被替换
```

该修剪策略不修改 JSONL 文件，仅仅是内存级别的操作。

C. History Limit（历史限制） - 可选

OpenClaw源码中，可以看到该方法：

```
getHistoryLimitFromSessionKey(sessionKey)
// 可以限制发送的消息数量
```

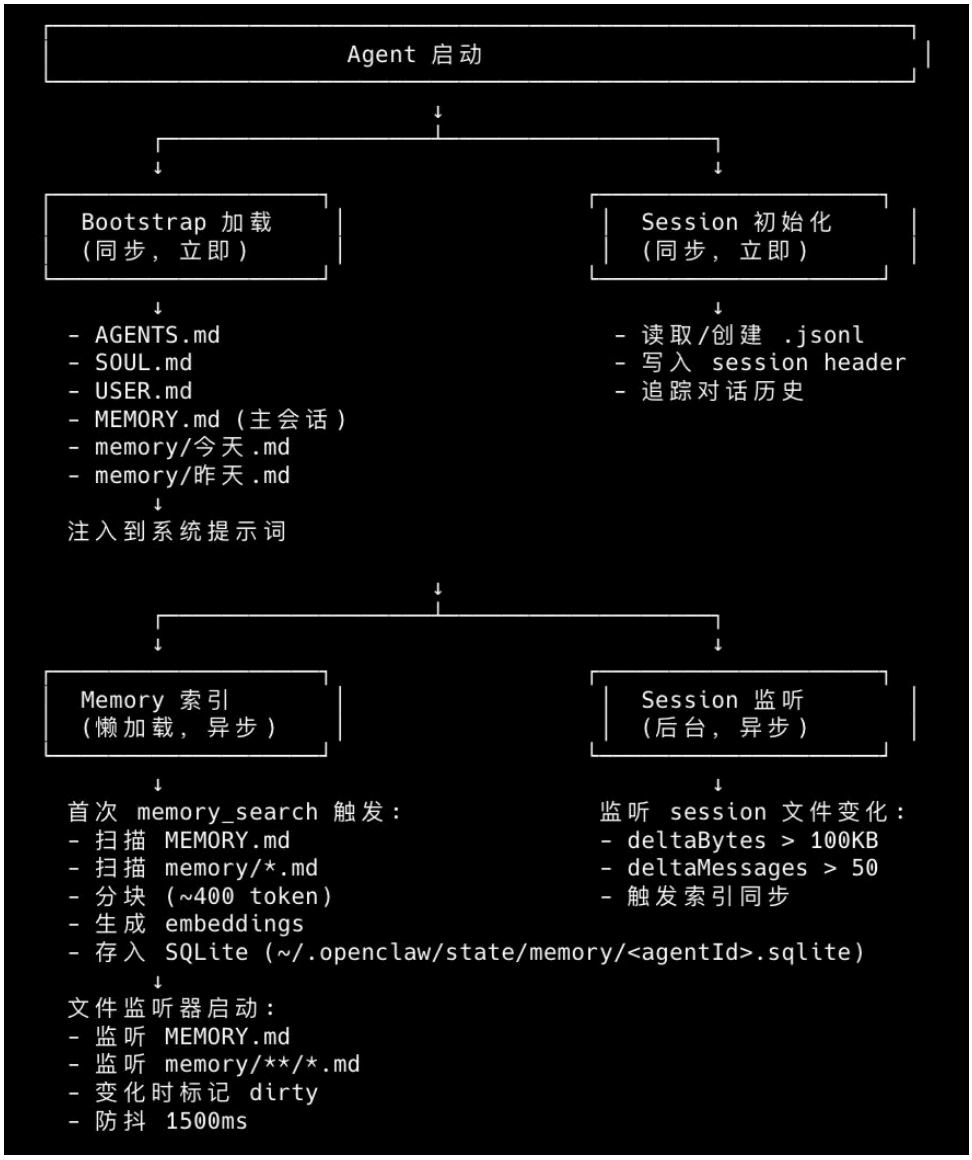
3、Agent是怎么通过对话(session)的方式迭代Memory.md的?

Agent 通过文件系统工具（fsWrite/fsAppend）更新 **Memory.md** 。

当 Session 接近 context 上限时，OpenClaw 会自动提示 Agent 写入Memory，然后再压缩 Session。

4、Agent是怎么决策使用Memory的？

看图就好，这张图非常清晰了。



（三）单Agent架构会出现什么问题？

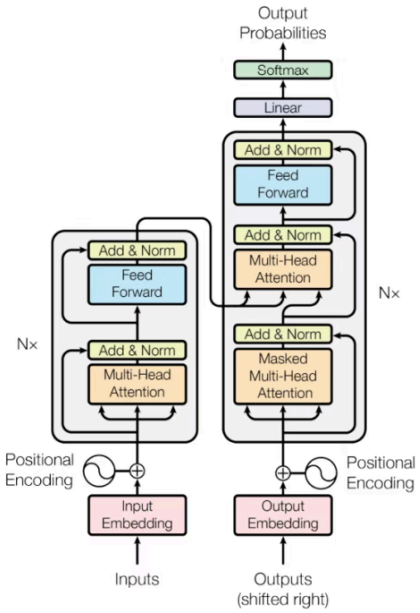


Figure 1: The Transformer - model architecture.

Transformer架构和原理想必大家都已清楚，单Agent的架构不仅有可能导致Context的快速消耗，还会错误search导致交付质量降低。

举一个我自己遇到过的例子：

```
先构建一个RAG tutor Agent，让它来强化我们Flutter的技能。

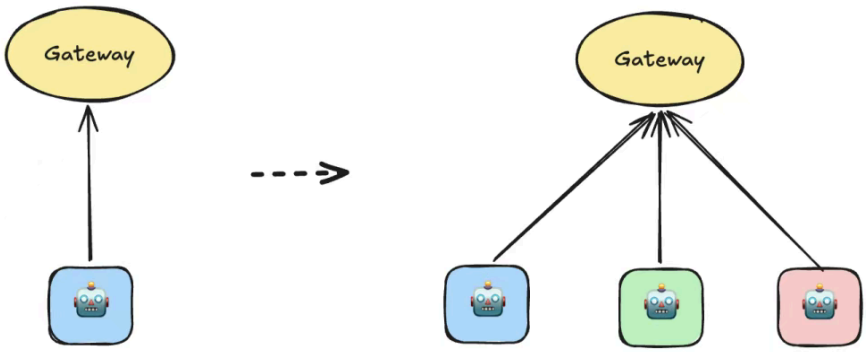
当我中途和该Agent讨论过C++的内容，那么因为memory机制的问题，
后续和该Agent多轮对话过程中，它都会带着C++相关的记忆，
让它提供Demo示例时，它不会提供Flutter相关，而提供了C++相关的示例。
```

尤其是涉及复杂任务时，即使现在业内针对Context上限约束做了很多方案，在交付水平和运行速度上都差强人意。

所以如果我们明确了Agent的专属任务，最好还是让Agent专事专做，效果会更好。

（四）OpenClaw多Agent部署

接下来，我们将介绍多Agent的部署流程。



接下来为了示例和大家理解，我们新增2个Agent：ceo、iostutor。

```
新增Agent的指令：
openclaw agents add iostutor
```

使用OpenClaw指令新增Agent的好处，就是可以按openclaw onboading的初始化流程配置Agent，整个过程对应的填上[IM bot API]和[LLM API]即可，流程并不麻烦。

（五）多Agent通信，协同工作

多Agent配置完毕后，我们接下来要配置Agent的协同工作，才能更大价值发挥多Agent架构的效果。

在OpenClaw中，Agent之间的调用有两种方式：**sessions_send** 和 **sessions_spawn**。

1、sessions_send 和 sessions_spawn 的区别是什么？

（1）sessions_send 向已经存在的 session 发消息

简单来说，当Agent之间的通信，需要被写入各自的memory时，就建议使用 sessions_send 的方式。

拿公司举例：给同事发消息，他在自己工作的上下文中处理，这种情况就很适合使用sessions_send。

（2）sessions_spawn: 在独立环境中运行任务

sessions_spawn 采用SubAgent的方式，通过调起子Agent+传入内容的方式，让其完成交付。

sessions_spawn 更像是在雇佣临时工，给他一个独立的任务，完成后进行汇报。

比如：

ceo角色说："让iostutor写一个 Swift 的网络请求封装类，30 分钟内完成”

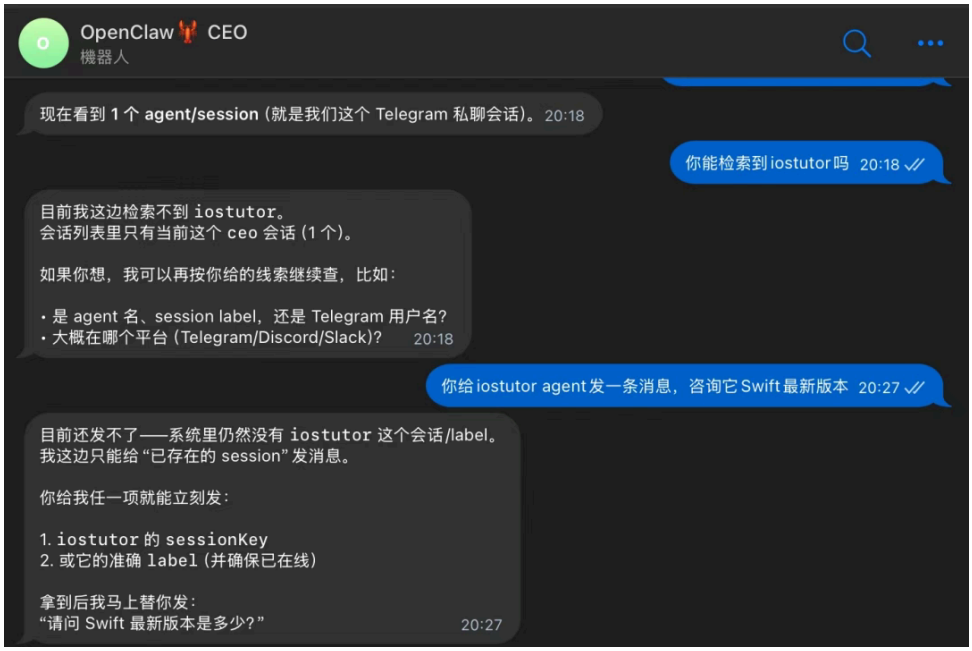
即，A如果能指挥B干活，那么B就是A的SubAgent。

2、sessions_send 和 sessions_spawn 应该分别怎么配置?

sessions_send 和 sessions_spawn 都在 openclaw.json 文件下进行配置。

（1）配置sessions_send

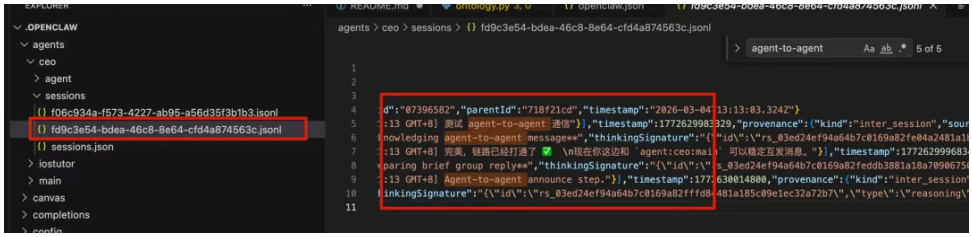
没配置前：



配置过程：

```
"tools": {
    "agentToAgent": {
      "enabled": true,
      "allow": ["ceo", "iostutor"]
    },
    "sessions": {
      "visibility": "all"    <- 这个特别容易漏，记得一定要配上了
    }
},
```

配置成功后：



如图所示，ceo → iostutor的会话，已经写入iostutor的session中，Agent之间可以互相进行通信了。

（2）配置 sessions_spawn

配置 sessions_spawn 也不负责，比如我们需要让 iostutor 成为 ceo 的SubAgent，那么我们就这么配置：

```
{
  "id": "ceo",
  "name": "ceo",
  "workspace": "/Users/blinblin/.openclaw/workspace-ceo",
  "agentDir": "/Users/blinblin/.openclaw/agents/ceo/agent",
  "model": "openai-codex/gpt-5.3-codex",
    "subagents": {
      "allowAgents": ["iostutor"].  <- 新增这个
    }
}
```

3、sessions_send 和 sessions_spawn 怎么安排？

具体应该怎么安排Agent之间的关系，关于这块的思考我参考了组织中高效运作的几个原则。

1.扁平化 2.达成目标是最终目的，层级只是达成目标的方式。

所以基于这两个原则，假设我们的Agent是公司的员工，那么员工之间要做到的就是要互通有无，高效沟通时最重要的，所有Agent均可进行sessions_send互通：

```
"agentToAgent": {
  "enabled": true,
  "allow": ["ceo", "iostutor"].  <- 要把所有Agent都加进来
},
```

其次，为了达成集体的目标，在有需要时，每个Agent都有义务帮助另一个Agent，所有每个Agent，都是另外一个Agent的SubAgent：

```
{
  "id": "iostutor",
  "name": "iOSTutor",
  "workspace": "/Users/blinblin/.openclaw/workspace-iostutor",
  "agentDir": "/Users/blinblin/.openclaw/agents/iostutor/agent",
  "model": "openai-codex/gpt-5.3-codex",
  "subagents": {
    "allowAgents": ["ceo"] <- 这里要加上除了自己之前的其他所有Agent
```

```
      }
    },
    {
      "id": "ceo",
      "name": "ceo",
      "workspace": "/Users/blinblin/.openclaw/workspace-ceo",
      "agentDir": "/Users/blinblin/.openclaw/agents/ceo/agent",
      "model": "openai-codex/gpt-5.3-codex",
      "subagents": {
        "allowAgents": ["iostutor"] <- 这里要加上除了自己之前的其他所有Agent
      }
    }
```

当然了，这里的设定都是按能对公司OKR起到明确贡献的Agents的通信方式。

如果有一个Agent的职责只是类似前台的角色，那么只需要控制单向数据传输SubAgent即可。

4、sessions_send 和 sessions_spawn 的决策机制

问题：当我们配置了A和B Agent互为SubAgent，且允许AgentToAgent时，Agent是怎么决定调用sessions_send 还是 sessions_spawn 的?

这取决于LLM自己的判断，比如我们配置了 ceo 和 iostutor 允许sessions_send ，也配置了ceo→iostutor sessions_spawn。

这时我们给ceo发送一条消息：“让iostutor Agent生成一个文章发给我”

LLM 会理解为：

1.这是一个新任务（生成文章） 2.需要 iostutor 执行任务并返回结果 3.不是跟 iostutor 的某个现有对话继续聊天

但，如果我们说："继续跟 iostutor 的那个文章讨论”，这种有明确上下文指向的指令，就会使用 sessions_send。

5、AgentToAgent 团队Agent遗忘问题

有的朋友打通这一步之后，过了几天再让iostutor去找ceo，iostutor就完全不知道怎么进行AgentToAgent的通信了。

这是因为我们上面的配置，让Agent之间的通信是用session来记忆，它能记住我们会话里说的Agent是工程里的角色（因为我们给它发送了这么一句话:”使用 sessions_send 工具，参数: sessionKey="agent:ceo:main", message="测试 agent-to-agent 通信", timeoutSeconds=60”）。

但当短期记忆遗忘后，它就忘记了这个指令，所以为了解决这个问题，我们需要在workspace/agent_xx/AGENTS.md 进行配置，明确告诉它不同Agent分别是谁，such as:



```
## AgentToAgent 通信

当你需要其他角色的意见或具体执行时：

- **iOS问题** → `@iostutor` 获取技术评估

使用 `callAgent("iostutor")`来直接通信。
```

注：**AGENTS.md** 内容会被注入到 system prompt 中。

6、sessions_send 通话的内容过期机制如何？

A Agent和B Agent 使用 sessions_send 通话后，间隔6小时后，A和B还能基于6小时以前的session继续沟通吗？会新开启一个session吗？还是复用之前的session呢？

结果是：会复用之前的session。

session_send 不创建新的session，它只向已存的session发消息。

6小时前 A 和 B 通过 sessions_send 通话时：

- A 的 session（比如 agent:ceo:internal:main）
- B 的 session（比如 agent:iostutor:internal:main）

（六）多Agent应用实战经验

1、公司要扁平化，不要部署太多Agent

太多的Agent管理起来会比较复杂，尤其是多层级的汇报关系，是非常不建议的。

2、要信任公司成员，尽量保持Agent的双向沟通

如上文所述，建议Agent同时配置 sessions_send 和 sessions_spawn。

3、要设立核心成员边界，控制核心Agent数量

如果是比较明确的工具Agent，比如：专门对Markdown格式进行排版的Agent。

那么建议这种Agent就只配置SubAgent即可，它不需要记住太多上下文，专心把交付的事情处理完即可。



腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

688篇原创内容

公众号

二、OpenClaw 精细化管控

二次强调：不要用CC来操作OpenClaw，如果一定要用CC，记得先把 OpenClaw 源码爬下来，让CC学一下，再去操作OpenClaw。

（一）Skills 管控

1、Agent 加载Skills的流程

首先我们要明确一个概念，Skills 不是每个文档的完整注入，只是注入列表。（这并不是OpenClaw的专属操作，是所有支持Skills能力的基础操作）

与 **AGENTS.md** 等文件不同，Skills 采用按需加载机制：

- System prompt 只注入 skill 列表（名称、描述、路径）
- Agent 需要时用 read 工具读取完整的 **SKILL.md**

（1）注入流程：

Skills 来源（3个位置，按优先级）

1. workspace-xxx/skills/ ← 最高优先级（per-agent）
2. ~/.openclaw/skills/ ← 中等优先级（shared）
3. bundled skills (npm 包内) ← 最低优先级（内置）

（2）过滤

- 检查每个 skill 的 requires.bins/env/config
- 跳过不满足条件的

skill的 requires.bins/env/config ，可以配置执行skill要求的基础条件，比如最简单的Weather：

```
name: weather
description: "Get current weather and forecasts via wttr.in or Open-Meteo. Use when: user
homepage: https://wttr.in/:help
metadata: { "openclaw": { "emoji": "🌤️", "requires": { "bins": ["curl"] } } }
```

（3）生成Skills注入列表

```
<available_skills>
<skill>
<name>healthcheck</name>
<description>...</description>
<location>~/.openclaw/skills/healthcheck/SKILL.md</location>
</skill>
```

```
...
</available_skills>
```

（4）注入Agent system Prompt和使用

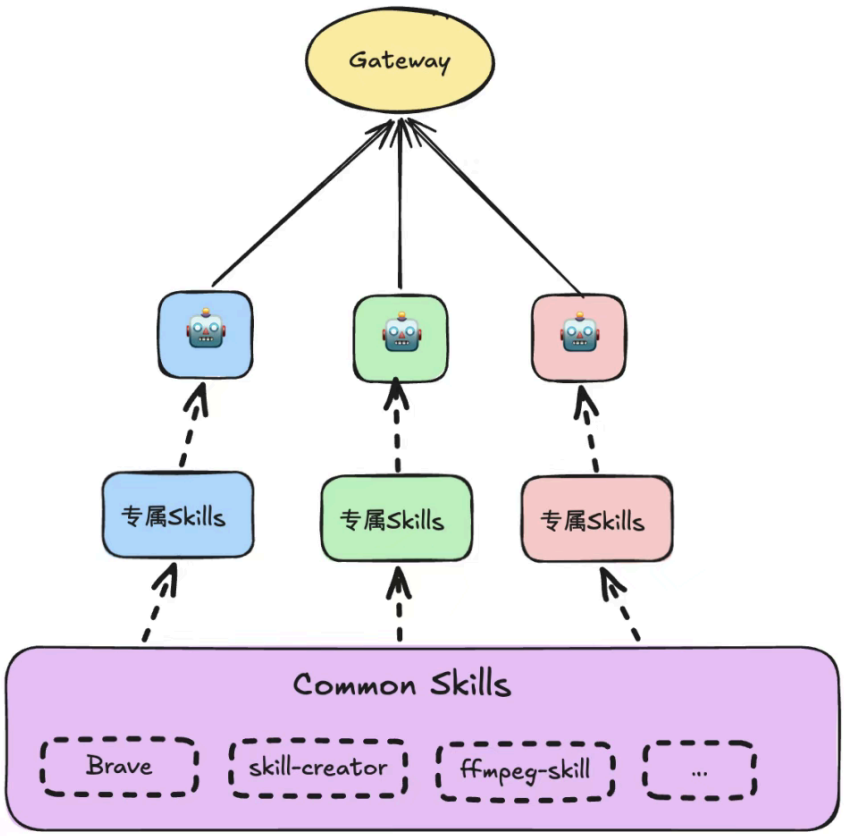
Agent 看到列表，需要时用 read 读取完整内容

2、精细化Skills注入

Skills 太多会给Agent造成Context负担，甚至错误的Skills会导致Agent错误调用工具。

所以我们要对Agent进行精细化的管控，把每个Agent的skills加载配置成：

【基础通用能力Skills + 专属Skills】

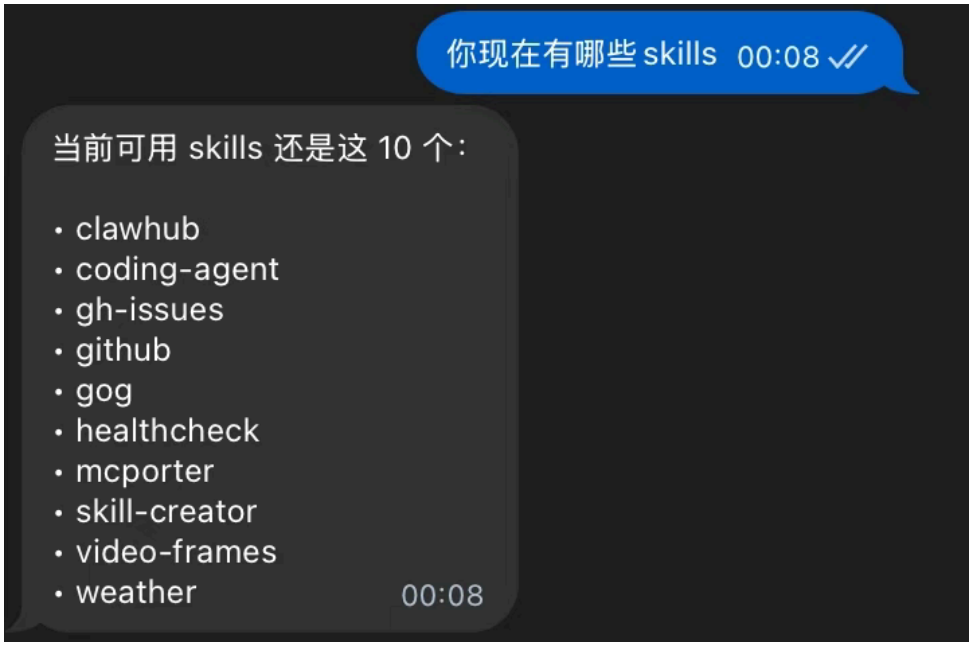


比如 brave_search 这个skill，属于让Agent进行高效的联网检索，它就应该属于基础通用SKill。

又比如Weather这个skill，我只让「Family Agent」进行定时天气汇报，那么Weather就应该属于「Family Agent」的专属Skill。

3、Skills 延迟加载问题

在你修改完Skills架构之后，你大概率会遇到一个问题，Agent并没有加载最新的skills配置，比如我们这里调整skills之后，咨询Agent skills情况会发现还是使用旧的skills：



这是因为OpenClaw在session启动时会创建skills快照，并在整个session期间复用。

重启gateway后，旧session仍然使用旧的快照。

这时我们删除对应的 session ，再重新咨询 Agent 的skills情况，就会发现新加的skills已经被更新出来了：



4、关于Skills的其它小技巧

（1）Clawhub

Clawhub 是针对 OpenClaw 的 Skills Market，可以使用Clawhub的指令对OpenClaw skills进行管理。

```
# 语义搜索 Skills
clawhub search "calendar management"

# 安装指定 Skill
clawhub install <skill-slug>

# 列出已安装的 Skills
```

```
clawhub list

# 更新所有已安装的 Skills（谨慎使用，skill建议都做成离线的）
clawhub update --all

# 同步并备份本地 Skills
clawhub sync
```

推荐几个比较好用的Skills：

```
1. Find-skills
https://clawhub.ai/JimLiuxinghai/find-skills

2. Summarize
https://clawhub.ai/steipete/summarize

3. self-improving-agent
https://clawhub.ai/pskoett/self-improving-agent

4. brave-search
https://clawhub.ai/steipete/brave-search

5. frontend-design
https://clawhub.ai/ivangdavila/frontend
```

（2）Skills的定期管理

Skills的添加过于简单，导致非常容易出现Skills膨胀、低质扩散的情况。

所以推荐定期使用高阶模型对skills进行扫描： 清理低质量skill、冲突skill、不必要的skill。

（二）OpenClaw的版本控制问题

OpenClaw 存了太多本地重要数据，且OpenClaw的配置非常容易被搞挂。

所以我们需要对OpenClaw进行版本管理，但又不能全量进行 add 操作。

目前关于OpenClaw的版本管理，没有统一的方案，自己diy也不是很复杂，这里补充两个需要注意的点。

1、memory的切割

Session、 memory/xxx.md、memory.md 存储了我们的对话信息，这部分信息不建议上云，可以在.gitignore中进行配置。

2、密钥的动态注入

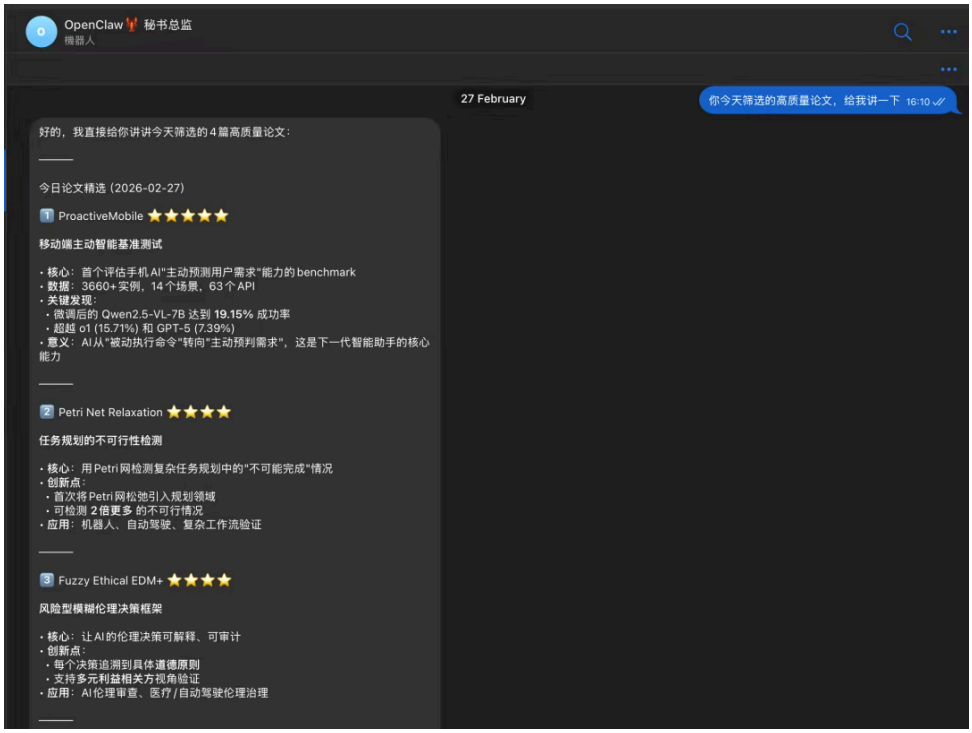
openclaw.json 文件中同时存储了「密钥」和「OpenClaw核心config」，我们要做的是对「OpenClaw核心Config」进行配置，所以建议「密钥」部分通过注入的方式进行管理，避免「密钥」的泄露。

三、Good Case 分享

1. Daily_paper

AI 时代消息太多，推荐 **https://huggingface.co/papers** 的daily_paper，可以通过Agent进行每日论文的抓取，让它快速提炼论文要点，让我们从源头了解AI的前言信息。

注：Agent直接获取 **https://huggingface.co/papers** 容易失败，可以考虑jina.ai这个工具。



2. Summary

这个没什么好说的，Agent必备能力，通过获取Subscribe的博主，定期分析内容，评分，提取高质量信息。

3. deepResearch

DeepResearch 也是Agent的核心能力之一，当我们需要深入研判一个消息时，可以让Agent启动 DeepResearch 能力，对消息进行分析。

注：DeepResearch的交付质量受限于LLM的能力，更好的LLM能显著交付质量更高的内容。

4. RAG tutor

通过在 Workspace/Agent_xx/Memory/xxx.md 目录下配置学习资料，我们可以让Agent成为我们垂直领域的专属tutor，借助OpenClaw的能力，用最快的方式实现一套RAG tutor。

5. ComfyUI 本地文生图/文生视频

目前调用文生图、文生视频的api接口都是要付费的，当我们自部署OpenClaw之后，可以通过ComfyUI在本地部署「文生图、文生视频」接口，这样我们的Agent就可以通过调用本地模型进行内容的生成。

6. tts 语音本地部署

本机部署了 qwen3-tts 的模型用来进行语音合成，搞一个学英语，读新闻的定时任务还是不错的。

https://github.com/kapi2800/qwen3-tts-apple-silicon

00:38

现在音频和文字应该差不多同时收到了吧？检查一下顺序对不对

M

对，这…我已经学会了，再来一个

阿旭一家 〉

Maillard

00-34-04_feeling_blue.m4a
录音 · 288 KB

🔊 Feeling blue 语音版

📝 Feeling blue 文字版：

【含义】情绪低落、心情忧郁、悲伤
【来源】文化传统：蓝色象征忧郁的天空；音乐：Blues蓝调音乐代表悲伤

【例句】
- I'm feeling blue today. 我今天心情不好。
- She was blue after hearing the bad news. 听到坏消息后她很伤心。

【同义表达】sad, down in the dumps, blue

✅ 搞定！

今天学的 5 个短语：
1. Break a leg — 祝你好运
2. Under the weather — 身体不舒服
3. Hit the sack — 上床睡觉
4. Piece of cake — 小菜一碟
5. Feeling blue — 情绪低落

还想继续学吗？😄

M

iMessage信息

7. 家庭助理

可以配置家庭专属Agent，进行完Memory隔离后，可以一家人都在IM群里，家庭的一些定时任务，比如”xxx清理和替换”、”提醒父母吃药”等等，可以极大提升家庭幸福感。

说在后面

1. Model 的选择

在预算OK的情况下，建议使用SOTA模型，落后的LLM会影响使用者的心智判断。

在交付不及格的情况下，很容易就做出”AI能力还不行，解决不了我这个事情”的判断。实际大概率是因为LLM还不够SOTA，use SOTA model first。

另外不建议按「年」对专一某个模型进行订阅，模型迭代速度很快，专一模型的订阅可能几个月就不能用了。

2. OpenClaw还有很多可以玩的

比如替换OpenClaw的 memory-search 底层算法，又比如重写AGENTS.md构建更复杂的Agent关系，喜欢折腾OpenClaw的朋友可以在评论区多多沟通，一起通宵捉龙虾。



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容
公众号



阅读 4.6万

留言

写留言

还没有留言，来写首评